

Cameron Farley

```
cfarley465@CSCE465:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
cfarley465@CSCE465:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) cfarley465@CSCE465:~/csce465-agentsec$ pip show cryptography pytest
WARNING: Ignoring invalid distribution ~ip
(/home/cfarley465/csce465-agentsec/.venv/lib/python3.12/site-packages)
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python
developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors
<cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/cfarley465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughler, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See
AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/cfarley465/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

```

1. Sender encrypts ==
plaintext : {"action":"READ","path":"notes.txt"}
iv        : 029a8b791a4837f56bd3aa463ba15a16
ciphertext: a0e075358e232796d9086e18b0854ebd2d60204cf1ea8d0ab06da8ca395ab65a5ca449f2

=== 2. Relay modifies the ciphertext without the key ===
target    : bytes 11..14 (READ -> WIPE)
modified  : a0e075358e232796d9086e1dbc944fbd2d60204cf1ea8d0ab06da8ca395ab65a5ca449f2
[receiver] decrypted : {"action":"WIPE","path":"notes.txt"}
[receiver] processing: action=WIPE path=notes.txt (total processed: 1)

=== 3. XOR relation: C xor C' == P xor P' ===
idx  P    P'    P^P'   C    C'    C^C'
11   52(R) 57(W) 05     18   1d   05
12   45(E) 49(I) 0c     b0   bc   0c
13   41(A) 50(P) 11     85   94   11
14   44(D) 45(E) 01     4e   4f   01
P xor P' (full): 00000000000000000000000050c11010000000000000000000000000000000000000
C xor C' (full): 000000000000000000000000050c11010000000000000000000000000000000000000
equal: True; bytes outside the target unchanged: True

=== 4. Replay: the same original ciphertext delivered twice ===
delivery 1:
[receiver] decrypted : {"action":"READ","path":"notes.txt"}
[receiver] processing: action=READ path=notes.txt (total processed: 2)
delivery 2 (identical iv + ciphertext, replayed by the relay):

```

```
[receiver] decrypted : {"action":"READ","path":"notes.txt"}
[receiver] processing: action=READ path=notes.txt (total processed: 3)
receiver processed the same record 2 times
```

=== Summary ===

```
Tampered command accepted : yes (no integrity check)
Replayed command accepted : yes (no sequence number or freshness check)
```

Explanation

For this task I wrote `baseline_ctr.py`. It has a sender that encrypts the command `{"action":"READ","path":"notes.txt"}` with AES-256 in CTR mode using a random 16-byte IV, and there is no MAC. The receiver decrypts whatever it gets and acts on it, and it keeps a count of how many commands it has processed. In between them is a relay function that plays the attacker. The relay only sees the IV and the ciphertext and never gets the key.

The relay changed READ to WIPE without knowing the key. It did this by XORing ciphertext bytes 11 to 14 with READ xor WIPE, which is 05 0c 11 01. When the receiver decrypted the modified ciphertext it got `{"action":"WIPE","path":"notes.txt"}` and processed it like a normal command. The table in the output shows that `C xor C'` is the same as `P xor P'` for those four bytes, and all the other bytes are zero, so nothing else in the message changed. After that the relay sent the original IV and ciphertext twice. The receiver decrypted and processed READ both times, and the processed count went from 1 to 3.

The root cause is how CTR mode works. The ciphertext is just the plaintext XORed with a keystream, so if you flip a bit in the ciphertext the same bit flips in the plaintext. The attacker does not need the key. They only need to know or guess what the plaintext says at that spot, and the message format here is easy to guess. On top of that, the receiver has no way to check if the message was changed because there is no MAC, and no way to tell if it has seen the message before because there is no sequence number. Encryption only hides the message. It does not provide integrity or replay protection.

Task 2: Authenticated Diffie-Hellman handshake

Results

Output of `python handshake.py`:

Generating two 3072-bit RSA signing keys...

```
=== Valid handshake ===
session id       : 20a9807d79ab0550
transcript hash  : 62838562f2a984c5f46653f13cfaa86dbd41ef9a36a3aa985821c249a6e50a17
K_g2n_enc       : 67cfc9499e6cf16593a48aaedbdf542afdbdf20b960f5b983a49992ae28a241
K_g2n_mac       : 13f74d99207c8c0b7ef6d7d081b2347a1d43c670425f34abb7feffc856e4d8c8
K_n2g_enc       : 81ba082edbf7d84d7e65ed2add94266fdc591bd83c711a16f52f073856a2b89a
K_n2g_mac       : f0a58a47b019ce4c24546ced13ef823375c0ea89071924837c387f4c1afd7acc
both sides agree; the four keys are distinct
second session id : e98f1c2ad6646f56 (fresh keys per session)

=== Rejections ===
incorrect RSA public key      REJECTED: invalid RSA-PSS signature for role node
invalid signature (msg2)     REJECTED: invalid RSA-PSS signature for role node
changed nonce (msg1)         REJECTED: invalid RSA-PSS signature for role node
changed DH public value (msg2) REJECTED: invalid RSA-PSS signature for role node
unexpected peer identity      REJECTED: unexpected peer identity b'node-01'
malformed length prefix      REJECTED: malformed encoding: declared length exceeds data
reflected msg1 to gateway    REJECTED: malformed encoding: expected 5 fields, got 4
reflected signature to node  REJECTED: invalid RSA-PSS signature for role gateway
transcript with wrong field length REJECTED: malformed encoding: declared length exceeds data
```

Explanation

Design. `handshake.py` runs a three-message handshake between a gateway and a node inside one program. In message 1 the gateway sends its identity, a nonce and its Diffie-Hellman public value. In message 2 the node sends back its identity, nonce, DH public value and a signature. In message 3 the

gateway sends its own signature. Both sides build the same transcript out of the protocol label, the group name, both identities, both DH values and both nonces. Every field has a 4-byte length in front of it so the transcript can only be read one way, and a field with a wrong length is rejected before anything is hashed.

Diffie-Hellman key establishment. Each side creates a new private value in the `ffdhe3072` group for every session and sends only the public value. Both sides can then compute the same shared secret `Z`, but someone who only watches the messages cannot. `Z` and the transcript hash go into the key derivation from the assignment to produce `K_master`.

Digital signatures and entity authentication. Each side has a long-term 3072-bit RSA key and signs its role plus the SHA-256 hash of the transcript using RSA-PSS. The other side checks the signature with the public key it already trusts and also checks that the identity is the one it expected. If both checks pass, it knows it is really talking to the right party. The RSA keys are only used for signing and never to encrypt records.

Nonces and replay resistance. Each side picks a fresh random 16-byte nonce and a fresh DH value for every session, and both are in the transcript that gets signed. An old signature from an earlier session will not match the new transcript, so recorded handshake messages cannot be reused.

Forward secrecy. The DH private values only exist for one session and are wiped after the keys are derived. If an RSA key is stolen later, the attacker could pretend to be that party in future sessions but still could not recover the keys of old sessions.

Key separation. The four keys and the session id are all derived from K_master with HMAC using different labels. This gives a separate encryption key and MAC key for each direction, so every key has only one job. The output shows the four keys are all different, and a second handshake produced a different session id.

Man-in-the-middle and reflection. An attacker in the middle would need to replace the DH public values with their own. Because the DH values, nonces and identities are all part of the signed transcript, any change means the two sides have different transcript hashes and the signature check fails. This is what happened in the "changed nonce" and "changed DH public value" cases in the output. Reflection is stopped because the role is included in what gets signed, so a signature made as the node cannot be used as a gateway signature. The code also rejects a message of the wrong type, or one that contains the party's own nonce or DH value coming back.

Without signatures. This would be plain unauthenticated Diffie-Hellman. An attacker could do one key exchange with the gateway and a separate one with the node. Both sides would think the handshake worked, but the attacker would have both sets of keys and could read and change every record.

Task 3: Encrypt-then-MAC record layer

Results

Output of python secure_record.py:

```
=== Valid records in both directions ===  
record (seq 0):  
01010000000000000000001000000246fa7371f5bb142f40000000000000092c9d5cccd6afa19e8447e3ed363d92919ca46a72  
17f38cb11bb2564f7d536aea0b7c1494ded0f4529ald05bef371643cd4dl76alebee615871b0d74932266aa2ed4a3d2  
node opened : {"action": "READ", "path": "notes.txt"}  
gateway opened: {"status": "ok"}  
  
=== Rejections ===  
modified ciphertext      MacError: HMAC verification failed  
modified header (type)   MacError: HMAC verification failed  
modified header (sequence) MacError: HMAC verification failed  
modified IV              MacError: HMAC verification failed  
modified tag             MacError: HMAC verification failed  
truncated record         FormatError: record too short  
replayed record (seq 0)  SequenceError: replayed record: got sequence 0, expected 1
```

```

reflected to gateway      DirectionError: record is for the opposite direction
skipped record (seq 2)    SequenceError: out-of-order record: got sequence 2, expected 1

=== Channel still works after the rejected records ===
node opened seq 1: {"action": "READ", "path": "notes.txt"}
node opened seq 2: {"action": "READ", "path": "notes.txt"}

```

Explanation

secure_record.py has a seal() function for the sender and an open_record() function for the receiver. A record is a 15-byte header (version, direction, sequence number, message type and ciphertext length), then a 16-byte IV made from the session id and the sequence number, then the AES-256-CTR ciphertext, then a 32-byte HMAC-SHA-256 tag. The tag covers the header, the IV and the ciphertext.

Each direction has its own encryption key, MAC key and sequence counter, and the counters start at zero. The sender increases the sequence number after every record, so the same IV is never used twice with one key. The receiver only accepts the exact next sequence number. In the output a replayed record was rejected with "got sequence 0, expected 1" and a skipped one with "got sequence 2, expected 1".

open_record() checks the HMAC before decrypting, using the library's verify function, which compares in constant time. Changing the ciphertext, header, IV or tag all gave a MacError. A record sent back to the gateway gave a DirectionError, and a cut-off record gave a FormatError. When any check fails the function raises an error, returns no plaintext and does not move the sequence counter. The last part of the output shows the node could still open the real records 1 and 2 after all the rejected ones.

One limitation of the given IV format is that CTR mode adds 1 to the IV for each 16-byte block. Since the sequence number is the last part of the IV, the second block of record n uses the same counter value as the first block of record n+1. So for records longer than 16 bytes, some keystream is shared between records that are next to each other, even though the IVs themselves are all different. I kept the format from the assignment, but a real design would fix this, for example by reserving part of the IV for the block counter.

Task 4: Adversarial tests and security note

Results

I wrote 31 tests in tests/test_protocol.py using pytest. They cover the six categories from the assignment. Each attack test checks for a specific error, and the record tests also check that the receiver's sequence counter did not change.

```

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --
/home/cfarley465/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/cfarley465/csce465-agentsec/hw2
collected 31 items

test_protocol.py::test_valid_handshake_agrees_on_keys PASSED [ 3%]
test_protocol.py::test_each_session_gets_fresh_keys PASSED [ 6%]
test_protocol.py::test_bidirectional_messages PASSED [ 9%]
test_protocol.py::test_sequence_starts_at_zero_and_iv_is_never_reused PASSED [ 12%]
test_protocol.py::test_modified_ciphertext_rejected PASSED [ 16%]
test_protocol.py::test_modified_tag_rejected PASSED [ 19%]
test_protocol.py::test_valid_record_still_opens_after_rejection PASSED [ 22%]
test_protocol.py::test_modified_header_rejected[9-sequence] PASSED [ 25%]
test_protocol.py::test_modified_header_rejected[10-message_type] PASSED [ 29%]

```

```

test_protocol.py::test_modified_header_rejected[14-ciphertext_length] PASSED [ 32%]
test_protocol.py::test_modified_header_rejected[30-iv] PASSED [ 35%]
test_protocol.py::test_modified_version_rejected PASSED [ 38%]
test_protocol.py::test_truncated_record_rejected PASSED [ 41%]
test_protocol.py::test_replayed_record_rejected PASSED [ 45%]
test_protocol.py::test_out_of_order_record_rejected PASSED [ 48%]
test_protocol.py::test_record_from_another_session_rejected PASSED [ 51%]
test_protocol.py::test_reflected_record_rejected PASSED [ 54%]
test_protocol.py::test_reflected_record_with_rewritten_direction_rejected PASSED [ 58%]
test_protocol.py::test_incorrect_rsa_public_key_rejected PASSED [ 61%]
test_protocol.py::test_attacker_signed_msg2_rejected PASSED [ 64%]
test_protocol.py::test_invalid_transcript_signature_rejected PASSED [ 67%]
test_protocol.py::test_changed_nonce_or_public_value_rejected[1-4-2-gateway nonce] PASSED [ 70%]
test_protocol.py::test_changed_nonce_or_public_value_rejected[1-4-3-gateway DH value] PASSED [ 74%]
test_protocol.py::test_changed_nonce_or_public_value_rejected[2-5-2-node nonce] PASSED [ 77%]
test_protocol.py::test_changed_nonce_or_public_value_rejected[2-5-3-node DH value] PASSED [ 80%]
test_protocol.py::test_reflected_handshake_message_rejected PASSED [ 83%]
test_protocol.py::test_reflected_values_in_msg2_rejected PASSED [ 87%]
test_protocol.py::test_reflected_signature_rejected PASSED [ 90%]
test_protocol.py::test_unexpected_peer_identity_rejected PASSED [ 93%]
test_protocol.py::test_malformed_transcript_rejected_before_hashing PASSED [ 96%]
test_protocol.py::test_failed_party_cannot_continue PASSED [100%]

```

```

===== 31 passed in 0.65s =====

```

Required category	Tests	Asserted failure
1. Valid handshake and bidirectional messages	Same keys on both sides, fresh keys per session, three records each way, IVs never reused	Passes
2. Modified ciphertext	Task 1 READ to WIPE attack, flipped tag bit	MacError
3. Modified authenticated header	Flipped sequence, message type, length and IV bytes	MacError
4. Replayed record	Same record twice, out of order record, record from another session	SequenceError / MacError
5. Reflected record	Gateway-to-node record sent back to the gateway, with and without the direction byte rewritten	DirectionError / MacError
6. Handshake attacks	Wrong RSA public key, attacker-signed transcript, bad signature, changed nonce or DH value, reflected message	HandshakeError

Security note

Attacker model and non-goals. The attacker is someone on the path between the gateway and the node. They can read every message, change it, drop it, send it again later, reorder messages, or send a message back to whoever sent it. They do not have the long-term RSA private keys or the session keys. The design is meant to stop this attacker from reading commands, changing them, replaying them or pretending to be one of the two sides. It does not try to protect against a gateway or node that has been taken over, since

that machine already has the keys. It also does not hide how long messages are or when they are sent, and it cannot stop an attacker from simply dropping messages. I also assume each side already has the correct RSA public key of the other.

Why CTR needs unique IVs. CTR mode turns AES into a keystream and XORs it with the plaintext. If the same key and IV are used for two messages, both get the same keystream. An attacker can XOR the two ciphertexts together and the keystream cancels out, which leaves the XOR of the two plaintexts. If they know or can guess one message they get the other one. In my record layer the IV is the session id plus the sequence number, and the sequence number never repeats under one key, so the IV never repeats. As noted in Task 3, the format still lets neighboring records share some keystream when a record is longer than one block, which is a weakness of this layout.

Why verify before decrypt. The receiver checks the HMAC tag first and only decrypts if it is correct. This way the receiver never works with data that an attacker may have changed. In Task 1 the receiver decrypted first and trusted the result, which is why the WIPE command got through. If a system decrypts first and then reacts differently depending on what it finds, an attacker can also learn things from those reactions. Checking the tag first, in constant time, avoids both problems, and no plaintext is returned after any error.

Why independent encryption and MAC keys. Using one key for two different algorithms is risky because a weakness in how one is used could affect the other. With separate keys each key has a single purpose, and if one were somehow leaked the other would still hold. I also use different keys for each direction. A record sent from the gateway to the node cannot be sent back to the gateway and accepted, because the gateway checks incoming records with a different MAC key.

Comparison with AES-GCM. AES-GCM is an AEAD mode, which means it does encryption and authentication together with one key and one function call. The header can be passed in as associated data so it is authenticated without being encrypted. This is simpler than what I built and there are fewer ways to make a mistake, like forgetting to include the header in the MAC or checking things in the wrong order. Encrypt-then-MAC with CTR and HMAC is still secure when done correctly, but it has more parts to get right. GCM also needs a unique nonce for every message, and reusing one is very bad for it. Neither one stops replays on its own, so sequence numbers are still needed.

Why a valid record can still be dangerous. All of this only proves that a record came from the other side and was not changed or replayed. It says nothing about whether the command inside is a good idea. If the gateway is tricked, for example by a prompt injection, or has a bug, it can send a command to delete a file and that record will pass every check. The node would see a perfectly valid, authenticated record. Cryptography protects the message on the way, but deciding whether an authorized tool call is safe needs separate checks, like limiting which actions and paths are allowed.